

WEEK 1 SUMMARY

Onboarding, C# Fundamentals, and Git

Program: Backend Development Internship (.NET)

Week: Week 1 of 10 | Phase 1: Onboarding and Foundations

Theme: .NET environment, core C#, object-oriented programming, collections, LINQ, and Git

Duration: 40 hours across five training days

Executive Summary

Week 1 established the development environment, language fundamentals, and professional workflow used throughout the .NET backend internship. Interns installed and verified the .NET SDK, configured an IDE, used the dotnet CLI, practiced core C# concepts, and closed the week with a GitHub feature-branch workflow and pull request.

The week intentionally focused on reliable foundations before introducing web frameworks. Types, nullable reference safety, object-oriented modeling, collections, LINQ, asynchronous programming, and clear Git practices are the building blocks for the ASP.NET Core APIs, database work, authentication, and capstone project that follow.

Learning Objectives Achieved

- Installed, configured, and verified a functional .NET SDK development environment.
- Used the dotnet CLI to create, build, and run a console application.
- Applied C# types, variables, control flow, and nullable reference types.
- Modeled a small domain using classes, records, interfaces, and OOP principles.
- Queried collections with LINQ and wrote basic async/await code.
- Used Git branches, commits, GitHub pushes, and pull requests.

Five-Day Curriculum and Practical Work

Day	Focus	Practical Outcome
1	Orientation & Environment	Verified .NET SDK, configured IDE, console application, GitHub setup.
2	C# Types & Control Flow	Types, nullable references, switch expressions, loops, and safe input.
3	Object-Oriented Programming	Classes, records, structs, encapsulation, interfaces, and

Day	Focus	Practical Outcome
		polymorphism.
4	Collections, LINQ & Async	Collection choice, LINQ queries, async/await, and exception handling.
5	Git & GitHub Workflow	Feature branch, purposeful commits, pull request, and mentor review.

Day 1: Orientation and .NET Development Environment

The program began with an overview of its phases, deliverables, evaluation process, mentor check-ins, Notion tracking, and future capstone options. The technical priority was a verified local environment: install the current .NET LTS SDK, confirm it is available through PATH with `dotnet --version` and `dotnet --list-sdks`, configure Visual Studio Code with C# Dev Kit or Visual Studio, and ensure IntelliSense and debugging work correctly.

Core CLI workflow: `dotnet new` console scaffolds an application; `dotnet build` compiles it; and `dotnet run` builds and executes it. The hands-on lab created HelloBinX, printed the intern's name and date, and created a personal GitHub repository.

Day 2: C# Types, Variables, and Control Flow

C# value types such as `int`, `bool`, and `struct` hold data directly and are copied by value. Reference types such as `class`, `string`, and arrays reference objects elsewhere in memory, so copying them copies the reference. Understanding this distinction prevents unintended shared-state behavior.

The program encouraged clear naming and pragmatic type inference: `var` is still strongly typed and works well when the assigned type is obvious, while public method signatures and return types should be explicit for readability. Control flow covered `if/else`, `switch` expressions, and `for`, `foreach`, and `while` loops.

Nullable reference types: With nullability enabled, `string?` explicitly allows null while `string` promises a non-null value. Compiler nullability warnings should be treated as real bug reports, not suppressed with the null-forgiving operator.

Day 3: Object-Oriented Programming

Classes, records, and structs were selected based on modeling needs. Classes suit entities with identity and behavior; records suit immutable, value-based data such as request and response DTOs; and structs suit small immutable values where copy-by-value makes sense. Encapsulation uses private fields and carefully chosen public properties and methods to protect valid state.

- Use inheritance only for a genuine is-a relationship.
- Use interfaces to define shared capabilities without forcing shared implementation.
- Favor interfaces over deep inheritance chains when types merely can do the same thing.

- Polymorphism lets code depend on an interface or base type while working with any valid implementation.

Day 4: Collections, LINQ, Async, and Exceptions

Collection choice should match access patterns: `List<T>` for general ordered data, `Dictionary<TKey, TValue>` for key-based lookup, and `HashSet<T>` when uniqueness matters. LINQ provides a declarative way to filter, project, sort, and aggregate collections through method syntax or query syntax.

Async/await was introduced for I/O-bound operations such as database, HTTP, and file access. Await releases the thread while work is pending, a necessity for scalable backend services. Exception handling should catch specific exceptions only when the code can recover or add useful context; avoid swallowing errors with broad, empty catch blocks.

Day 5: Git and GitHub Workflow

Git records a project as a sequence of meaningful commits. The regular loop is git add to stage, git commit to create a snapshot, and git push to synchronize with GitHub. Work is performed on a feature branch rather than directly on main so the main branch remains stable and deployable.

Professional standard: Commit messages should be specific and use imperative verbs such as Add, Fix, or Refactor. A pull request should explain what changed, why it changed, and any open questions so a mentor can review it efficiently.

Required Deliverables

- Verified .NET SDK environment and configured IDE.
- Console program showing types, control flow, and nullable-reference handling.
- Small OOP domain model with classes, records, and at least one interface.
- LINQ queries and an async method operating on a collection.
- GitHub repository with a feature branch, clear commits, and an open pull request.
- Week 1 summary ready for mentor check-in.

Testing and Review Checklist

1. Run `dotnet --version` and successfully build and run the console application.
2. Demonstrate value versus reference copy behavior and safe nullable-input handling.
3. Verify the domain model protects valid state and demonstrates interface-based polymorphism.
4. Run filter, projection, and aggregation LINQ queries; await an asynchronous operation; test meaningful exception handling.
5. Confirm the feature branch is pushed, pull request is descriptive, and mentor review is requested.